Documento de apoyo (Sesión 3, Unidad 2)

Procesos ETL y Modelado de Datos para Análisis OLAP

Fundamentos del proceso ETL y su aplicación en OLAP

Javier Andrés Valdez González / Auxiliar

Seminario de Sistemas 2 — Sección P

Universidad de San Carlos de Guatemala

Febrero de 2026

Resumen

El proceso ETL (Extract, Transform, Load) constituye el mecanismo operativo central para consolidar datos procedentes de sistemas heterogéneos hacia repositorios analíticos, como bodegas de datos (Data Warehouses) y estructuras multidimensionales destinadas a OLAP (Online Analytical Processing). Este documento desarrolla, con enfoque técnico–académico, la definición de ETL, sus fases, las actividades de extracción, transformación y carga, sus características esperables y su rol específico en la construcción y mantenimiento de modelos OLAP. Asimismo, se discuten desafíos recurrentes (calidad de datos, cargas incrementales, evolución de esquemas, ventanas de carga y observabilidad) y se presenta un caso práctico con ejemplos en Python (pandas) para cargar datos en un esquema dimensional sobre Microsoft SQL Server.

Palabras clave: ETL, OLAP, Data Warehouse, SQL Server, pandas, CDC, calidad de datos, trazabilidad.

Índice

1. Introducción **4**

2. Fundamentos del proceso ETL y su aplicación en OLAP **5**
2.1. ¿Qué es ETL? . . . 5
2.2. OLAP y su relación con ETL . . . 5
2.3. ETL vs. ELT (contexto arquitectónico) . . . 5

3. Fases del proceso ETL **6**

4. Extracción, Transformación y Carga **7**
4.1. Extracción . . . 7
4.2. Transformación . . . 7
4.3. Carga . . . 8

5. Características del ETL (orientado a OLAP) **9**
5.1. Reproducibilidad e idempotencia . . . 9
5.2. Trazabilidad, auditoría y observabilidad . . . 9
5.3. Manejo explícito de errores . . . 9
5.4. Escalabilidad y rendimiento . . . 9
5.5. Seguridad y conectividad . . . 9

6. Rol del ETL en OLAP **10**
6.1. Definición de OLAP y cubos . . . 10
6.2. Cómo ETL habilita OLAP . . . 10
6.3. Procesamiento de cubos como fase final . . . 10
6.3.1. Objetivos del procesamiento . . . 10

7. Desafíos del proceso ETL **12**
7.1. Calidad de datos y heterogeneidad semántica . . . 12
7.2. Cargas incrementales y captura de cambios . . . 12
7.3. Evolución de esquemas (schema drift) . . . 12
7.4. Ventanas de carga y rendimiento . . . 12
7.5. Observabilidad y recuperación ante fallos . . . 12
7.6. Orquestación y calendarización . . . 12

8. Caso práctico: hoja de trabajo 3 (SQL Server + Python) 13
8.1. Modelo dimensional de referencia (esquema estrella) 13
8.2. DDL (SQL Server) para dimensiones y hechos 13
8.3. Instrucciones para reproducibilidad del ejemplo 14
8.4. DDL completo reproducible (SQL Server) 15

9. Ejemplos en Python (pandas) para cargar datos y actualizar un cubo SQL Server 20
9.1. Conectividad a SQL Server (ODBC / mssql-python) 20
9.2. Ejemplo 1: extracción y transformación mínima con pandas 20
9.3. Ejemplo 2: conexión a SQL Server con ODBC Driver 18 21
9.4. Ejemplo 3: carga de dimensiones (incremental simple) y hechos 21
9.5. Ejemplo 4: consumo incremental con CDC (conceptual) 23

10. Tabla de síntesis: desafíos y mitigaciones 25

11. Conclusiones 26

1. Introducción

Las organizaciones modernas generan datos en múltiples aplicaciones operacionales: ventas, inventario, contabilidad, logística, atención al cliente y canales digitales. Estos sistemas transaccionales están optimizados para registrar eventos con integridad (ACID), baja latencia y reglas específicas del proceso operativo. Sin embargo, la toma de decisiones gerencial requiere consolidar información a través de áreas, comparar periodos, evaluar tendencias y analizar métricas a diferentes niveles de detalle (por ejemplo, por día, por mes, por región o por categoría de producto). Esta diferencia entre necesidades operacionales y analíticas obliga a construir una arquitectura que integre, estandarice y preserve el significado corporativo de los datos.

El proceso ETL (Extract, Transform, Load) se utiliza para integrar datos desde fuentes heterogéneas hacia un repositorio analítico unificado. En su definición general, ETL extrae datos, los transforma (limpieza, estandarización, integración, enriquecimiento) y los carga en un destino, comúnmente una bodega de datos o un data mart [3]. Desde la perspectiva de arquitectura de bodegas de datos, ETL es crítico porque materializa reglas de negocio en transformaciones repetibles, establece la granularidad de los hechos, construye claves sustitutas y conforma dimensiones [6].

En el contexto de OLAP, ETL cobra relevancia adicional: OLAP se fundamenta en el análisis multidimensional y en la agregación consistente (sumarizabilidad). Un modelo OLAP (cubo multidimensional o tabular) depende de datos conformados y consistentes para producir resultados confiables en operaciones como *slice*, *dice*, *drill-down* y *roll-up* [1], [5]. Por ello, el objetivo de este documento es presentar un marco conceptual y práctico de ETL orientado a análisis OLAP y a un flujo reproducible apoyado en SQL Server como destino analítico y en Python (pandas) como herramienta de extracción, transformación y carga. [3], [6], [15]

2. Fundamentos del proceso ETL y su aplicación en OLAP

2.1. ¿Qué es ETL?

ETL es un proceso de integración de datos compuesto por tres actividades esenciales: extracción, transformación y carga. En términos funcionales, (i) la extracción obtiene datos desde sistemas fuente; (ii) la transformación aplica reglas de negocio, limpieza, estandarización e integración; y (iii) la carga inserta o actualiza datos transformados en un repositorio objetivo para consumo analítico [3]. En una bodega de datos, ETL no debe entenderse como un mero *pipeline* técnico, sino como el mecanismo por el cual se construye consistencia corporativa: al definir catálogos, jerarquías y métricas, el ETL formaliza el significado de los datos [6].

2.2. OLAP y su relación con ETL

OLAP (Online Analytical Processing) describe tecnologías y enfoques para consultas analíticas de alta velocidad y análisis multidimensional sobre grandes volúmenes de datos [5]. Codd et al. propusieron criterios y componentes para la categoría OLAP, enfatizando la perspectiva de los usuarios analistas y el enfoque multidimensional [1]. En OLAP, una métrica (por ejemplo, ventas netas) se analiza a través de dimensiones (tiempo, producto, geografía, canal). El ETL es el responsable de garantizar que dichas dimensiones estén conformadas, que las claves sean consistentes y que las métricas se calculen bajo reglas reproducibles [6].

2.3. ETL vs. ELT (contexto arquitectónico)

En arquitecturas modernas, también existe ELT (Extract, Load, Transform), donde los datos se cargan primero al destino y las transformaciones se ejecutan principalmente allí. IBM distingue ETL de ELT, señalando que difieren en el orden operativo y en el aprovechamiento del motor de destino [4]. En entornos SQL Server, ambos patrones pueden coexistir: el staging puede cargarse de forma temprana (L) y las transformaciones pesadas pueden implementarse como procedimientos T-SQL (T), manteniendo trazabilidad y control. En cursos OLAP, la elección debe justificarse con base en rendimiento, gobernanza, control de calidad y capacidades del motor analítico [6].

3. Fases del proceso ETL

Aunque cada organización adapta su implementación, un ETL robusto suele estructurarse en fases (o subprocesos) que facilitan control, auditoría y operación:

F1. Planeación y análisis de fuentes: inventario, diccionario de datos, perfiles de calidad, definición de reglas de negocio y granularidad [6].

F2. Extracción: conectividad a fuentes, definición de cargas completas vs. incrementales, manejo de ventanas y control de marcas de agua [3], [14].

F3. Staging: aterrizaje temporal que desacopla fuentes del destino, facilita re-procesos y auditoría [6].

F4. Transformación: limpieza, estandarización, deduplicación, conformado, validaciones y enriquecimiento; manejo de errores por fila y cuarentena de registros inválidos [6].

F5. Carga al modelo dimensional: inserción/actualización de dimensiones y hechos, gestión de historización y consistencia de claves [6].

F6. Auditoría y monitoreo: bitácoras de ejecución, métricas operacionales, reconciliación, alertas y orquestación con calendarización [6].

En una implementación reproducible orientada a laboratorio, estas fases pueden implementarse con scripts de Python (pandas) y procedimientos SQL, manteniendo parametrización, bitácoras y controles de calidad.

4. Extracción, Transformación y Carga

4.1. Extracción

La extracción se centra en obtener datos desde fuentes sin afectar la operación. Sus estrategias comunes incluyen:

- **Extracción completa (Full Load):** replica el conjunto total; útil para dimensiones pequeñas o fuentes sin mecanismos de cambios [3].
- **Extracción incremental:** obtiene solo cambios desde la última ejecución (por timestamp, identificadores crecientes o CDC) [3].
- **CDC (Change Data Capture) en SQL Server:** captura inserciones, actualizaciones y eliminaciones, exponiéndolas en tablas de cambios que se consultan en formato relacional [14].

Un diseño incremental confiable requiere *marcas de agua*, ventanas temporales y consistencia en re-ejecuciones. Por ejemplo, si se extrae por timestamp, se debe considerar latencia de escritura, zonas horarias, y el caso de registros atrasados (*late arriving data*) [6].

4.2. Transformación

La transformación es el componente semántico del ETL: es donde se materializan reglas de negocio y se construye consistencia corporativa. Entre tareas típicas se incluyen:

T1. Limpieza y estandarización: normalización de tipos, formatos, unidades y codificaciones (p. ej., fechas ISO, moneda, mayúsculas/minúsculas) [6].

T2. Validación de dominio: rangos permitidos, obligatoriedad de campos críticos, integridad de catálogos [6].

T3. Deduplicación y resolución de entidades: consolidación de registros duplicados con reglas reproducibles [6].

T4. Conformado de dimensiones: mapeo de códigos distintos a un catálogo común, construcción de jerarquías coherentes [6].

T5. Manejo de errores por fila: separar fallos recuperables (datos sucios) de fallos no recuperables (infraestructura), registrando el detalle del error y redirigiendo los registros inválidos a tablas de cuarentena para su corrección [6].

Desde una perspectiva OLAP, la transformación debe asegurar sumarizabilidad: que los totales agregados sean coherentes al cambiar niveles jerárquicos. formalizan el operador *cube* como generalización N-dimensional de agregaciones, subrayando la importancia de agregaciones consistentes [2].

4.3. Carga

La carga inserta o actualiza datos transformados en el destino analítico. Patrones de carga:

- **Append:** insertar nuevos hechos; útil si los hechos no se corrigen retroactivamente.
- **Upsert:** insertar/actualizar; típico en dimensiones y en hechos con correcciones.
- **Full refresh:** reemplazo total; útil en catálogos pequeños o snapshots.

En cuanto a historización, el enfoque dimensional suele usar SCD. Adicionalmente, SQL Server ofrece tablas temporales con versionado de sistema para mantener historial de cambios y habilitar consultas *point-in-time* [12].

5. Características del ETL (orientado a OLAP)

Un ETL orientado a OLAP debería exhibir las siguientes características:

5.1. Reproducibilidad e idempotencia

El pipeline debe poder re-ejecutarse sin duplicar hechos ni distorsionar dimensiones. Esto implica controles de llaves, staging persistente y reglas deterministas [6].

5.2. Trazabilidad, auditoría y observabilidad

La observabilidad del ETL se apoya en bitácoras de ejecución (inicio/fin, duración), conteos por etapa, marcas de agua y registros de error por lote o por fila. Estas prácticas habilitan auditoría, troubleshooting y re-ejecución controlada [6].

5.3. Manejo explícito de errores

En un enfoque con Python/pandas, estas decisiones se implementan mediante validaciones previas, conversión controlada de tipos y escritura de registros fallidos a tablas de cuarentena con el motivo y la columna causante [6].

5.4. Escalabilidad y rendimiento

Se recomiendan cargas por lotes, paralelismo controlado, índices adecuados en staging, y técnicas de carga masiva cuando sea necesario [6].

5.5. Seguridad y conectividad

La conectividad segura hacia SQL Server depende de drivers y configuraciones. Microsoft documenta el ODBC Driver para SQL Server y su descarga/uso como base de conectividad estándar para aplicaciones [8], [9].

6. Rol del ETL en OLAP

6.1. Definición de OLAP y cubos

IBM define OLAP como tecnología para consultas complejas de alta velocidad o análisis multidimensional sobre grandes volúmenes de datos [5]. Establecieron criterios y componentes para la categoría OLAP, describiendo arquitectura habilitadora y criterios de evaluación [1].

6.2. Cómo ETL habilita OLAP

ETL es condición necesaria para un OLAP confiable porque:

- **Establece granularidad:** define exactamente qué representa una fila de hechos.
- **Construye dimensiones conformadas:** evita comparaciones inconsistentes entre áreas o periodos.
- **Asegura sumarizabilidad:** previene dobles conteos, jerarquías rotas y agregaciones contradictorias [2].

6.3. Procesamiento de cubos como fase final

En un flujo ETL orientado a OLAP, la carga de dimensiones y hechos no suele ser el último paso operativo. Para que el *cubo* (entendido aquí como la estructura que habilita análisis multidimensional) quede listo para consulta, se requiere una fase de **procesamiento** que actualice estructuras analíticas derivadas: agregaciones, índices, particiones y validaciones de consistencia.

Desde el punto de vista conceptual, el cubo se apoya en agregaciones a múltiples niveles (por ejemplo, ventas por día/mes, por producto/categoría, por tienda/región). El operador *cube* formaliza este tipo de agregación N-dimensional como una generalización del *group-by* relacional, lo que sustenta el uso de agregados precomputados para acelerar consultas OLAP. [2], [5]

6.3.1. Objetivos del procesamiento

La fase de procesamiento persigue cuatro objetivos principales:

P1. Actualizar dimensiones: incorporar cambios en atributos y jerarquías (p. ej., nuevos productos o tiendas, recategorizaciones) para que el análisis por filtros y niveles jerárquicos sea coherente.

P2. Sincronizar hechos: incorporar nuevas transacciones y, cuando aplique, correcciones retroactivas sin duplicar eventos.

P3. Materializar estructuras de acceso rápido: (re)construir agregaciones e índices (y, si se utiliza, particionamiento temporal) para reducir el costo de consulta.

P4. Verificar consistencia: asegurar sumarizabilidad (totales consistentes entre niveles) y ausencia de llaves huérfanas o duplicidades. [2], [6]

7. Desafíos del proceso ETL

Esta sección resume desafíos frecuentes y sus implicaciones en OLAP.

7.1. Calidad de datos y heterogeneidad semántica

Las fuentes no comparten necesariamente definiciones ni catálogos. Un mismo concepto ("cliente", "venta neta") puede variar entre sistemas. ETL debe transformar dicha heterogeneidad en reglas explícitas y versionadas [3], [6].

7.2. Cargas incrementales y captura de cambios

Las cargas completas pueden ser inviables por volumen. CDC facilita incrementales en SQL Server al registrar cambios y exponerlos en tablas/funciones para consumo [14]. Aun así, hay retos: retención de cambios, reconciliación, reintentos y consistencia temporal [6].

7.3. Evolución de esquemas (schema drift)

Cambios en columnas, tipos o códigos rompen pipelines si no hay contratos y validaciones. Se recomiendan pruebas automatizadas, versionado y monitoreo de esquema [6].

7.4. Ventanas de carga y rendimiento

Se deben cumplir ventanas operativas. En grandes volúmenes, la estrategia incluye micro-batches, particionamiento, bulk loads y paralelismo controlado [6].

7.5. Observabilidad y recuperación ante fallos

Los fallos son inevitables. Un pipeline reproducible debe aislar errores por fila (calidad) y por infraestructura, y registrar evidencias para re-procesar sin duplicar datos [6].

7.6. Orquestación y calendarización

La dependencia dims $\rightarrow$ facts $\rightarrow$ cubo debe automatizarse mediante un planificador (épocas de carga, reintentos, alertas) respetando el orden de dependencias y verificaciones posteriores [6].

8. Caso práctico: hoja de trabajo 3 (SQL Server + Python)

8.1. Modelo dimensional de referencia (esquema estrella)

Se propone un ejemplo minimalista para OLAP:

- **DimFecha(FechaKey, Fecha, Anio, Mes, Dia, Trimestre)**
- **DimProducto(ProductoKey, SKU, Nombre, Categoria)**
- **DimTienda(TiendaKey, Codigo, Nombre, Region)**
- **FactVentas(FechaKey, ProductoKey, TiendaKey, Cantidad, PrecioUnitario, Descuento, TotalNeto)**

Separar dimensiones (atributos descriptivos) de hechos (medidas) facilita agregaciones coherentes y mejora el rendimiento analítico [6].

8.2. DDL (SQL Server) para dimensiones y hechos

Listing 1: DDL simplificado para un esquema estrella

```
CREATE SCHEMA dwh;
GO

CREATE TABLE dwh.DimFecha (
  FechaKey     INT         NOT NULL PRIMARY KEY,
  Fecha        DATE        NOT NULL,
  Anio         SMALLINT    NOT NULL,
  Mes          TINYINT     NOT NULL,
  Dia          TINYINT     NOT NULL,
  Trimestre    TINYINT     NOT NULL
);

CREATE TABLE dwh.DimProducto (
  ProductoKey  INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  SKU          VARCHAR(50) NOT NULL UNIQUE,
  Nombre       NVARCHAR(200) NOT NULL,
  Categoria    NVARCHAR(100) NULL
```

```
);

CREATE TABLE dwh.DimTienda (
  TiendaKey    INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  Codigo       VARCHAR(50) NOT NULL UNIQUE,
  Nombre       NVARCHAR(200) NOT NULL,
  Region       NVARCHAR(100) NULL
);

CREATE TABLE dwh.FactVentas (
  FechaKey       INT NOT NULL,
  ProductoKey    INT NOT NULL,
  TiendaKey      INT NOT NULL,
  Cantidad       INT NOT NULL,
  PrecioUnitario DECIMAL(18,2) NOT NULL,
  Descuento      DECIMAL(18,2) NOT NULL DEFAULT(0),
  TotalNeto      AS (Cantidad * PrecioUnitario - Descuento)
     PERSISTED,
  CONSTRAINT PK_FactVentas PRIMARY KEY (FechaKey, ProductoKey,
     TiendaKey),
  CONSTRAINT FK_FactVentas_Fecha  FOREIGN KEY (FechaKey) REFERENCES
     dwh.DimFecha(FechaKey),
  CONSTRAINT FK_FactVentas_Prod   FOREIGN KEY (ProductoKey)
     REFERENCES dwh.DimProducto(ProductoKey),
  CONSTRAINT FK_FactVentas_Tienda FOREIGN KEY (TiendaKey) REFERENCES
      dwh.DimTienda(TiendaKey)
);
GO
```

8.3. Instrucciones para reproducibilidad del ejemplo

Para que el ejemplo en Python sea **ejecutable de extremo a extremo** (sin depender de objetos previos), se recomienda la siguiente secuencia:

1. Ejecutar el script DDL completo `ddl_reproducible_dwh.sql` en SQL Server (SSMS o Azure Data Studio). Este script:
   - crea (opcionalmente) la base de datos `SGFoodDW`,
   - crea el esquema `dwh`,

- crea las tablas `DimFecha`, `DimProducto`, `DimTienda`, `FactVentas`,
- crea la vista analítica `dwh.v_CuboVentas` (base relacional para cubo/modelo tabular),
- crea índices recomendados.

2. Verificar conectividad (ODBC Driver 18) y credenciales.

3. Ejecutar el script `cargar_ejemplo_reproducible.py`. Si el archivo `ventas_sample.csv` no existe, el script genera uno automáticamente con semilla fija para reproducibilidad.

8.4. DDL completo reproducible (SQL Server)

El siguiente script consolida el DDL necesario para reproducir el ejemplo completo (incluye base de datos opcional, esquema, tablas, vista y índices):

Listing 2: DDL reproducible: SGFoodDW + esquema estrella + vista analítica + índices

```
/*
  DDL reproducible para el ejemplo ETL -> OLAP (SQL Server)
  Objetivo:
   1) Crear (opcional) la base de datos SGFoodDW
   2) Crear el esquema dwh
   3) Crear DimFecha, DimProducto, DimTienda, FactVentas
   4) Crear una vista analítica dwh.v_CuboVentas (base para cubo /
      modelo tabular)
   5) Crear índices sugeridos para acelerar agregaciones

  Nota:
   - Si ya existe SGFoodDW, se reutiliza.
   - Si tu usuario no tiene permisos para CREATE DATABASE, crea la
     BD manualmente y ejecuta desde USE SGFoodDW.
*/

-- 1) Crear base de datos (opcional)
IF DB_ID(N'SGFoodDW') IS NULL
BEGIN
  CREATE DATABASE SGFoodDW;
END
GO

USE SGFoodDW;
```

```
GO

-- 2) Crear esquema
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'dwh')
  EXEC(N'CREATE SCHEMA dwh');
GO

-- 3) Tablas Dimensionales
IF OBJECT_ID(N'dwh.DimFecha', N'U') IS NULL
BEGIN
  CREATE TABLE dwh.DimFecha (
    FechaKey     INT         NOT NULL PRIMARY KEY,   -- formato YYYYMMDD
    Fecha        DATE        NOT NULL,
    Anio         SMALLINT    NOT NULL,
    Mes          TINYINT     NOT NULL,
    Dia          TINYINT     NOT NULL,
    Trimestre    TINYINT     NOT NULL,
    CONSTRAINT CK_DimFecha_Mes CHECK (Mes BETWEEN 1 AND 12),
    CONSTRAINT CK_DimFecha_Dia CHECK (Dia BETWEEN 1 AND 31),
    CONSTRAINT CK_DimFecha_Trimestre CHECK (Trimestre BETWEEN 1 AND 4)
  );
END
GO

IF OBJECT_ID(N'dwh.DimProducto', N'U') IS NULL
BEGIN
  CREATE TABLE dwh.DimProducto (
    ProductoKey  INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    SKU          VARCHAR(50)       NOT NULL,
    Nombre       NVARCHAR(200)     NOT NULL,
    Categoria    NVARCHAR(100)     NULL,
    CONSTRAINT UQ_DimProducto_SKU UNIQUE (SKU)
  );
END
GO

```

```
59 IF OBJECT_ID(N'dwh.DimTienda', N'U') IS NULL
60 BEGIN
61   CREATE TABLE dwh.DimTienda (
62     TiendaKey    INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
63     Codigo       VARCHAR(50)       NOT NULL,
64     Nombre       NVARCHAR(200)     NOT NULL,
65     Region       NVARCHAR(100)     NULL,
66     CONSTRAINT UQ_DimTienda_Codigo UNIQUE (Codigo)
67   );
68 END
69 GO
70
71 -- 4) Tabla de hechos
72 IF OBJECT_ID(N'dwh.FactVentas', N'U') IS NULL
73 BEGIN
74   CREATE TABLE dwh.FactVentas (
75     FechaKey       INT           NOT NULL,
76     ProductoKey    INT           NOT NULL,
77     TiendaKey      INT           NOT NULL,
78     Cantidad       INT           NOT NULL,
79     PrecioUnitario DECIMAL(18,2) NOT NULL,
80     Descuento      DECIMAL(18,2) NOT NULL CONSTRAINT
         DF_FactVentas_Descuento DEFAULT(0),
81     TotalNeto      AS (Cantidad * PrecioUnitario - Descuento)
         PERSISTED,
82
83     CONSTRAINT PK_FactVentas PRIMARY KEY (FechaKey, ProductoKey,
         TiendaKey),
84     CONSTRAINT FK_FactVentas_Fecha   FOREIGN KEY (FechaKey)
         REFERENCES dwh.DimFecha(FechaKey),
85     CONSTRAINT FK_FactVentas_Prod    FOREIGN KEY (ProductoKey)
         REFERENCES dwh.DimProducto(ProductoKey),
86     CONSTRAINT FK_FactVentas_Tienda  FOREIGN KEY (TiendaKey)
         REFERENCES dwh.DimTienda(TiendaKey),
87
88     CONSTRAINT CK_FactVentas_Cantidad CHECK (Cantidad >= 0),
89     CONSTRAINT CK_FactVentas_PrecioUnitario CHECK (PrecioUnitario >=
         0),
```

```
    CONSTRAINT CK_FactVentas_Descuento CHECK (Descuento >= 0)
  );
END
GO

-- 5) Vista analítica (base relacional para cubo / modelo tabular)
IF OBJECT_ID(N'dwh.v_CuboVentas', N'V') IS NOT NULL
  DROP VIEW dwh.v_CuboVentas;
GO

CREATE VIEW dwh.v_CuboVentas
AS
SELECT
  f.FechaKey,
  df.Fecha,
  df.Anio,
  df.Mes,
  df.Dia,
  df.Trimestre,
  f.ProductoKey,
  dp.SKU,
  dp.Nombre     AS ProductoNombre,
  dp.Categoria  AS ProductoCategoria,
  f.TiendaKey,
  dt.Codigo     AS TiendaCodigo,
  dt.Nombre     AS TiendaNombre,
  dt.Region     AS TiendaRegion,
  f.Cantidad,
  f.PrecioUnitario,
  f.Descuento,
  f.TotalNeto
FROM dwh.FactVentas AS f
JOIN dwh.DimFecha    AS df ON f.FechaKey    = df.FechaKey
JOIN dwh.DimProducto AS dp ON f.ProductoKey = dp.ProductoKey
JOIN dwh.DimTienda   AS dt ON f.TiendaKey   = dt.TiendaKey;
GO

-- 6) Índices sugeridos
```

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'
   IX_FactVentas_Fecha' AND object_id = OBJECT_ID(N'dwh.FactVentas')
   )
BEGIN
  CREATE NONCLUSTERED INDEX IX_FactVentas_Fecha
    ON dwh.FactVentas(FechaKey)
    INCLUDE (Cantidad, TotalNeto);
END
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'
   IX_FactVentas_Producto' AND object_id = OBJECT_ID(N'dwh.
   FactVentas'))
BEGIN
  CREATE NONCLUSTERED INDEX IX_FactVentas_Producto
    ON dwh.FactVentas(ProductoKey)
    INCLUDE (Cantidad, TotalNeto);
END
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'
   IX_FactVentas_Tienda' AND object_id = OBJECT_ID(N'dwh.FactVentas'
   ))
BEGIN
  CREATE NONCLUSTERED INDEX IX_FactVentas_Tienda
    ON dwh.FactVentas(TiendaKey)
    INCLUDE (Cantidad, TotalNeto);
END
GO

-- 7) Consulta de validación
-- SELECT TOP (20) * FROM dwh.v_CuboVentas ORDER BY Fecha DESC;
```

9. Ejemplos en Python (pandas) para cargar datos y actualizar un cubo SQL Server

9.1. Conectividad a SQL Server (ODBC / mssql-python)

Para Python, dos enfoques comunes:

- **ODBC + SQLAlchemy (pyodbc):** requiere el ODBC Driver de Microsoft [8], [9].
- **mssql-python:** driver promovido por Microsoft con DDBC y DB-API 2.0 [10], [11].

pandas permite leer y escribir con `read_sql` y `DataFrame.to_sql` [15], [16].

9.2. Ejemplo 1: extracción y transformación mínima con pandas

Supóngase un CSV `ventas.csv` con columnas: `fecha, sku, tienda_codigo, cantidad, precio_unit, descuento`. El siguiente fragmento ilustra lectura y limpieza.

Listing 3: Extracción y transformación mínima (pandas)

```
import pandas as pd

df = pd.read_csv("ventas.csv")

# Tipos y limpieza básica
df["fecha"] = pd.to_datetime(df["fecha"], errors="coerce").dt.date
df["sku"] = df["sku"].astype(str).str.strip()
df["tienda_codigo"] = df["tienda_codigo"].astype(str).str.strip()

for c in ["cantidad", "precio_unit", "descuento"]:
    df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0)

# Reglas de dominio (ejemplo): fecha no nula y cantidad no negativa
df = df[(df["fecha"].notna()) & (df["cantidad"] >= 0)].copy()

# Métrica derivada
df["total_neto"] = df["cantidad"] * df["precio_unit"] - df["
   descuento"]
```

9.3. Ejemplo 2: conexión a SQL Server con ODBC Driver 18

Listing 4: Conexión con SQLAlchemy + ODBC Driver 18

```
import urllib
from sqlalchemy import create_engine

server = "MI_SERVIDOR"
database = "MI_BD"
username = "MI_USUARIO"
password = "MI_PASSWORD"

params = urllib.parse.quote_plus(
    "DRIVER={ODBC Driver 18 for SQL Server};"
    f"SERVER={server};DATABASE={database};UID={username};PWD={
       password};"
    "Encrypt=yes;TrustServerCertificate=yes;"
)

engine = create_engine(f"mssql+pyodbc:///?odbc_connect={params}")
```

9.4. Ejemplo 3: carga de dimensiones (incremental simple) y hechos

Este ejemplo didáctico inserta solo nuevos SKUs y tiendas, construye DimFecha y luego carga hechos resolviendo claves sustitutas.

Listing 5: Carga de DimProducto, DimTienda, DimFecha y FactVentas

```
import pandas as pd

# --- DimProducto y DimTienda: insertar solo nuevos valores ---
dim_prod = df[["sku"]].drop_duplicates().rename(columns={"sku":"SKU"
   })
dim_prod["Nombre"] = dim_prod["SKU"]   # placeholder
dim_prod["Categoria"] = None

dim_tienda = df[["tienda_codigo"]].drop_duplicates().rename(columns
   ={"tienda_codigo":"Codigo"})
dim_tienda["Nombre"] = dim_tienda["Codigo"]  # placeholder
dim_tienda["Region"] = None
```

```
11
12 prod_exist = pd.read_sql("SELECT SKU FROM dwh.DimProducto", engine)
13 tienda_exist = pd.read_sql("SELECT Codigo FROM dwh.DimTienda",
       engine)
14
15 dim_prod_new = dim_prod[~dim_prod["SKU"].isin(prod_exist["SKU"])]
16 dim_tienda_new = dim_tienda[~dim_tienda["Codigo"].isin(tienda_exist[
       "Codigo"])]
17
18 if not dim_prod_new.empty:
19     dim_prod_new.to_sql("DimProducto", engine, schema="dwh",
           if_exists="append", index=False)
20
21 if not dim_tienda_new.empty:
22     dim_tienda_new.to_sql("DimTienda", engine, schema="dwh",
           if_exists="append", index=False)
23
24 # --- DimFecha: generar para el rango observado ---
25 fechas = pd.Series(pd.to_datetime(df["fecha"]).unique()).sort_values
       ()
26 cal = pd.DataFrame({"Fecha": pd.to_datetime(fechas)})
27 cal["FechaKey"] = cal["Fecha"].dt.strftime("%Y%m%d").astype(int)
28 cal["Anio"] = cal["Fecha"].dt.year.astype(int)
29 cal["Mes"] = cal["Fecha"].dt.month.astype(int)
30 cal["Dia"] = cal["Fecha"].dt.day.astype(int)
31 cal["Trimestre"] = cal["Fecha"].dt.quarter.astype(int)
32 cal["Fecha"] = cal["Fecha"].dt.date
33
34 exist = pd.read_sql("SELECT FechaKey FROM dwh.DimFecha", engine)
35 cal_new = cal[~cal["FechaKey"].isin(exist["FechaKey"])]
36
37 if not cal_new.empty:
38     cal_new[["FechaKey","Fecha","Anio","Mes","Dia","Trimestre"]].
           to_sql(
39         "DimFecha", engine, schema="dwh", if_exists="append", index=
               False
40     )
41
```

```
# --- Hechos: resolver claves sustitutas y cargar ---
dim_prod_map = pd.read_sql("SELECT ProductoKey, SKU FROM dwh.DimProducto", engine)
dim_tienda_map = pd.read_sql("SELECT TiendaKey, Codigo FROM dwh.DimTienda", engine)

fact = df.copy()
fact["FechaKey"] = pd.to_datetime(fact["fecha"]).dt.strftime("%Y%m%d").astype(int)

fact = fact.merge(dim_prod_map, left_on="sku", right_on="SKU", how="left")
fact = fact.merge(dim_tienda_map, left_on="tienda_codigo", right_on="Codigo", how="left")

# Validación: no permitir llaves faltantes
if fact["ProductoKey"].isna().any() or fact["TiendaKey"].isna().any():
    missing = fact[fact["ProductoKey"].isna() | fact["TiendaKey"].isna()]
    raise ValueError(f"Existen llaves faltantes. Ejemplo:\n{missing.head()}")

fact_out = fact[["FechaKey","ProductoKey","TiendaKey","cantidad","precio_unit","descuento"]].rename(
    columns={"cantidad":"Cantidad","precio_unit":"PrecioUnitario","descuento":"Descuento"}
)

fact_out.to_sql("FactVentas", engine, schema="dwh", if_exists="append", index=False)
```

9.5. Ejemplo 4: consumo incremental con CDC (conceptual)

CDC en SQL Server captura cambios y los expone en funciones/tablas de cambios [7], [14]. Un consumo conceptual:

Listing 6: Consulta conceptual a CDC (rango de LSN)

```
DECLARE @from_lsn binary(10) = sys.fn_cdc_get_min_lsn('dbo_Ventas');
```

```
DECLARE @to_lsn   binary(10) = sys.fn_cdc_get_max_lsn();

SELECT *
FROM cdc.fn_cdc_get_all_changes_dbo_Ventas(@from_lsn, @to_lsn, 'all'
   );
```

Listing 7: Lectura del delta CDC con pandas.read_sql (conceptual)

```
import pandas as pd

sql = """
DECLARE @from_lsn binary(10) = sys.fn_cdc_get_min_lsn('dbo_Ventas');
DECLARE @to_lsn   binary(10) = sys.fn_cdc_get_max_lsn();
SELECT *
FROM cdc.fn_cdc_get_all_changes_dbo_Ventas(@from_lsn, @to_lsn, 'all
   ');
"""
delta = pd.read_sql(sql, engine)

# delta suele incluir metadata (p.ej., __$operation) para aplicar ló
   gica de upsert/delete
print(delta.head())
```

10. Tabla de síntesis: desafíos y mitigaciones

Desafío	**Riesgo en OLAP**	**Mitigación recomendada**
Calidad de datos heterogénea [3]	Métricas inconsistentes, jerarquías rotas	Perfilamiento, reglas explícitas, cuarentena de errores, auditoría [6]
Cargas incrementales [14]	Latencia alta o duplicación de hechos	CDC / marcas de agua, idempotencia, reconciliación, monitoreo [13]
Evolución de esquemas	Fallos de carga, columnas inesperadas	Versionado, validación previa, contratos de datos, pruebas automatizadas [6]
Ventanas de carga	No disponibilidad del cubo o retraso	Particiones, micro-batches, bulk loads, paralelismo controlado [6]
Observabilidad insuficiente [6]	Difícil detectar fallos y causas	Logging, métricas operacionales, alertas, bitácoras por etapa [6]

11. Conclusiones

ETL es el componente operativo que transforma datos dispersos en un repositorio analítico confiable. Su importancia en OLAP radica en que define granularidad, conforma dimensiones, asegura sumarizabilidad y formaliza reglas de negocio de forma reproducible [2], [6]. En un entorno basado en SQL Server, mecanismos incrementales como CDC pueden apoyar la detección de cambios para alimentar el modelo analítico, mientras que Python/pandas puede implementarse como capa reproducible de extracción, transformación y carga [14], [15].

Los ejemplos en Python con pandas ilustran una alternativa práctica para prototipos, automatizaciones complementarias o cargas ligeras: se extraen datos, se aplican transformaciones mínimas y se cargan dimensiones y hechos con `to_sql`. En un entorno productivo, se recomienda fortalecer el pipeline con staging persistente, auditoría, pruebas automatizadas, seguridad y orquestación robusta; no obstante, el flujo conceptual se mantiene: *extraer* $\rightarrow$ *transformar* $\rightarrow$ *cargar* $\rightarrow$ *validar*.

Referencias

[1] E. F. Codd, S. B. Codd y C. T. Salley, «Providing OLAP to User-Analysts: An IT Mandate,» E. F. Codd & Associates, inf. téc., 1993, White paper. dirección: https://staff.icar.cnr.it/manco/Teaching/2005/datamining/articoli/olapcoddwp.pdf

[2] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow y H. Pirahesh, «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals,» *Data Mining and Knowledge Discovery*, vol. 1, n.º 1, págs. 29-53, 1997. DOI: 10.1023/A:1009726021843

[3] IBM. «What is ETL (Extract, Transform, Load)?» Visitado 6 de feb. de 2026. dirección: https://www.ibm.com/think/topics/etl

[4] IBM. «What Is Extract, Load, Transform (ELT)?» Visitado 6 de feb. de 2026. dirección: https://www.ibm.com/think/topics/elt

[5] IBM. «What is OLAP?» Visitado 6 de feb. de 2026. dirección: https://www.ibm.com/think/topics/olap

[6] R. Kimball y J. Caserta, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley, 2004.

[7] Microsoft. «Change Data Capture Functions (Transact-SQL),» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/relational-databases/system-functions/change-data-capture-functions-transact-sql

[8] Microsoft. «Download ODBC Driver for SQL Server,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/connect/odbc/download-odbc-driver-for-sql-server

[9] Microsoft. «Microsoft ODBC Driver for SQL Server,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/connect/odbc/microsoft-odbc-driver-for-sql-server

[10] Microsoft. «Microsoft Python Driver for SQL Server - mssql-python,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/connect/python/mssql-python/python-sql-driver-mssql-python

[11] Microsoft. «Quickstart: Connect with the mssql-python driver for Python,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/connect/python/mssql-python/python-sql-driver-mssql-python-quickstart

[12] Microsoft. «Temporal Tables - SQL Server,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables

[13] Microsoft. «Track Data Changes - SQL Server,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/track-data-changes-sql-server

[14] Microsoft. «What is change data capture (CDC)? - SQL Server,» visitado 6 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/about-change-data-capture-sql-server

[15] pandas development team. «pandas.DataFrame.to_sql — pandas documentation,» visitado 6 de feb. de 2026. dirección: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_sql.html

[16] pandas development team. «pandas.read_sql — pandas documentation,» visitado 6 de feb. de 2026. dirección: https://pandas.pydata.org/docs/reference/api/pandas.read_sql.html